PROJECT-BASED LEARNING

PHASE-I EVALUATION

Inverted Index and Code Based Search Engine

Team ID:DSCPP-III-2026-T112

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T112
Semester: 3rd
Section: A
Domain: *CSE*

Team Members

1. *Anmol Gupta – 2510385219*
2. *Harshit Kumar – 2510012451*
3. *Atharva Goyal – 2510360047*
4. *Navneet Nainwal – 2510010627*

Mentor

(Prof.)Dr.Jyoti Agarwal

Interactions completed: ___ Yes ___

Problem Statement & Motivation

Problem Statement

The Inverted Index and Code-Based Search Engine solves problem by creating an inverted index that maps keywords to the files and locations where they appear. Allows users to search for keywords and quickly retrieve relevant files without scanning entire dataset.To develop a fast, organized and efficient search engine that reduces search time and demonstrates the practical application of DSA concepts. Clearly describe the real-world problem, its context, and the specific gap that the project intends to address.

Motivation

- *Fast Information Retrieval*
- *Efficient Data Organization*
- *Scalable Search System*

Target Users / Stakeholders

- *Students and Researchers*
- *Developers and Programmers*
- *Organizations and Educational Institutions*

Objectives

Project Objectives

1. *Implement an inverted index for code search.*
2. *Apply hashing for fast keyword retrieval.*
3. *Use data structures for efficient index management.*
4. *Implement searching and file-indexing techniques.*
5. *Analyze time and space complexity.*
6. *Develop an efficient code search engine.*

Key Objective: *To enable fast and efficient searching of large codebases using Inverted Indexing, Hashing, and Data Structures.*

Proposed Solution

Proposed Approach

1. *Document Input*
2. *Text Tokenization*
3. *Inverted Indexing*
4. *Query Processing*
5. *Ranked Output*

Key Features

- *Fast Word Search*
- *Document Mapping*
- *Efficient Storage*
- *Ranked Results*
- *File-Based Indexing*

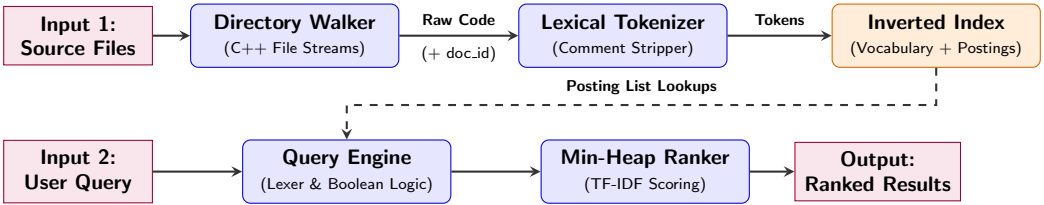
The proposed solution uses an inverted index to map each word from PDF/TXT documents to the files where it appears. This allows users to search keywords quickly without scanning every document each time.It improves search speed, accuracy, and document retrieval efficiency.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	*Arrays, Linked Lists, Trees, Graphs, Hashing, etc.*	*Inverted Index and Data Management*
OOPs with C++	*Classes, Inheritance, Polymorphism, STL, etc.*	*Search and Query Processing*
Operating Systems	*Processes, Scheduling, Memory, Synchronization, etc.*	*File and Memory Management*
DBMS	*SQL, Transactions, Normalization, Indexing, etc.*	*Indexing and Data Storage*

Requirement: DSA, C++, Operating System concepts are combined to efficiently process, organize and search documents.

System Architecture / Workflow



Major Components

- *Parses raw code and extracts clean lowercase tokens.*
- *Pure C dynamic engine linking words to heap-allocated posting lists.*
- *Recursive search module retrieving matching document nodes.*

Data / Control Flow

- *Code Files→Tokens→Inverted Index→Query Match*
- *Tokenizer feeds words into the C index; Query Engine returns results.*
- *Matched Doc IDs and term counts with zero memory leaks.*

Technology Stack & Methodology

Technology Stack

- Programming: *C, C++, JavaScript, HTML, CSS*
- Database: *Text Files / File Handling*
- Tools / Framework: *STL, DOM, C/C++ Library*
- IDE: *VS Code*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Evaluation

C/C++ with STL guarantees high-speed execution and precise memory control, which is essential for building the core inverted index data structures. Text-file data handling provides a lightweight, modular database approach that perfectly aligns with the file stream parsing requirements. A JavaScript/DOM frontend bridges the robust backend logic with an intuitive interface for seamless, real-time user query interaction.

Initial Progress & Team Contribution

Progress Achieved

- *Background studied*
- *Requirements identified*
- *Architecture prepared*
- *PDF/TXT Planned*
- *Index Logic Designed*

Individual Contribution

Member	Role	Contribution
Anmol Gupta	*Lead*	*Authored the project report, contributed to core logic development, conducted system debugging and managed project coordination.*
Harshit Kumar	*Developer*	*Developed backend logic in C/C++, implemented the frontend interface using HTML/CSS/JS and conducted technical research.*
Navneet Nainwal	*DB/Testing*	*Executed comprehensive system testing, performed bug tracking and resolution and assisted in foundational research.*
Atharva Goyal	*Developer/Docs*	*Designed the project presentation, managed documentation, and collaborated on backend code development.*

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Development
3. Phase-III: Final Implementation

Expected Outcomes

- *Working Search Prototype*
- *Fast Search Results*
- *Document Search Utility*
- *Scalable Index System*

References

1. *The C++ Programming Language(4th Edition)-Bjanre Strouptrup*
2. *Mastering Algorithms with C-Kyle Loudon(O'Reilly Media)*
3. *https://nlp.stanford.edu/IR-book/*
4. *https://www.elastic.co/guide/en/elasticsearch/reference/current/documents-indices.html*

Thank You
Questions & Suggestions